

Tracing Provenance and Detecting Tampering with Complementary LLM Watermarks

Xiaoyan Feng, Yanjun Zhang, He Zhang, Leo Yu Zhang, and Shirui Pan
Griffith University

Abstract—Watermarking LLM-generated text is an important task for tracing its provenance. Existing LLM watermarks preserve provenance under editing, but this same robustness allows an adversary to alter critical content while retaining attribution, a vulnerability known as piggyback spoofing. We introduce an innovative watermark that jointly provides provenance and tamper evidence. It co-embeds a robust signal and a fragile signal into each generated token. The signals share the same mechanism but use independent keys and different seeding windows over normalized text, making one resilient to edits and the other sensitive to reader-visible changes. Multiple rounds of unbiased tournament reweighting preserve the expected generation distribution, while a periodic round-allocation pattern controls the trade-off between the two signals. At detection, their scores form a two-dimensional space supporting three decisions: Intact, Tampered, and No-Watermark. Across two large language models and two prompt datasets, our method demonstrates the highest tamper-detection rate among the evaluated methods while maintaining competitive attribution robustness and perplexity. Ablation studies show that reliable three-state detection requires a well-defined notion of intactness, co-embedding of the two signals, and complementary sensitivity to edits.

Index Terms—large language models, text watermarking, provenance, tamper evidence

I. INTRODUCTION

Regulations and industry commitments increasingly require AI-generated content to be identifiable [1]–[3]. Generative watermarking is a principled path to this requirement, tracing the *provenance* of LLM-generated text [4], [5]. It tilts the sampling probabilities toward tokens favored by a keyed pseudorandom signal, and later scores how strongly the received text follows this signal. The first threat to watermarking is removal, in which an attacker edits part of the watermarked text yet still evades provenance attribution. Existing schemes therefore keep the signal robust to edits to ensure reliable provenance [6], [7]. This robustness opens up a new attack surface. An adversary may alter critical details of watermarked text while keeping the signal strong, making the scheme endorse content the model did not generate. This attack is called *piggyback spoofing* [8]. Figure 2 illustrates both attacks on the same green-red watermark [4]. Countering piggyback spoofing requires *tamper evidence*. Tamper evidence calls for a signal sensitive to edits, which is the opposite of what provenance demands. Yet most schemes embed a single signal and thus face a dilemma.

Embedding two signals in one text decouples provenance and tamper evidence and promises both. A watermark scheme fulfilling this goal requires a tailored signal design, embedding

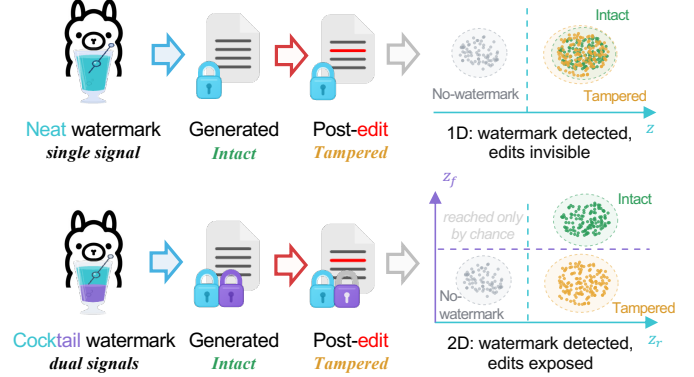


Fig. 1. Cocktail co-embeds two signals into one text: a **robust signal** and a **fragile signal**. A lightly edited text keeps a high robust score but drops to a low fragile score, making tamper evident while preserving provenance.

mechanism, and detection rule. The closest attempt, Bileve [9], pays a price at every stage. It signs the first tokens of a generation and writes the signature bits into the following tokens. The carrier tokens are generated after the signature is computed and cannot enjoy its protection. At detection, its signature fails even on unedited text, because re-tokenization of the delivered text may shift the token IDs. Generation quality also degrades, with perplexity rising about sixfold, because the sampler takes the top-ranked token whose hash matches the signature bit.

We propose Cocktail, which embeds two complementary signals that differ only in the key and the seeding window, achieving provenance and tamper evidence simultaneously. The signal for provenance is robust to edits via a short window, while the signal for tamper evidence is fragile to edits via a long window. The signals are seeded on normalized text to ensure they react to reader-visible edits but stay consistent during transmission. Both signals are co-embedded into each token, so that every token contributes to both demands. The co-embedding is implemented by vectorized tournament reweighting, with a periodic pattern controlling the signal strength ratio while maintaining generation quality. In detection, the two signal scores span a 2D plane where Intact, Tampered, and No-Watermark text occupy separable regions, as Figure 1 previews. A two-threshold rule then reads the three states directly off the plane. Our contributions are fourfold.

- We resolve the dilemma by decoupling: a robust and a fragile signal in one text serve provenance and tamper evidence separately, and their joint score turns the binary

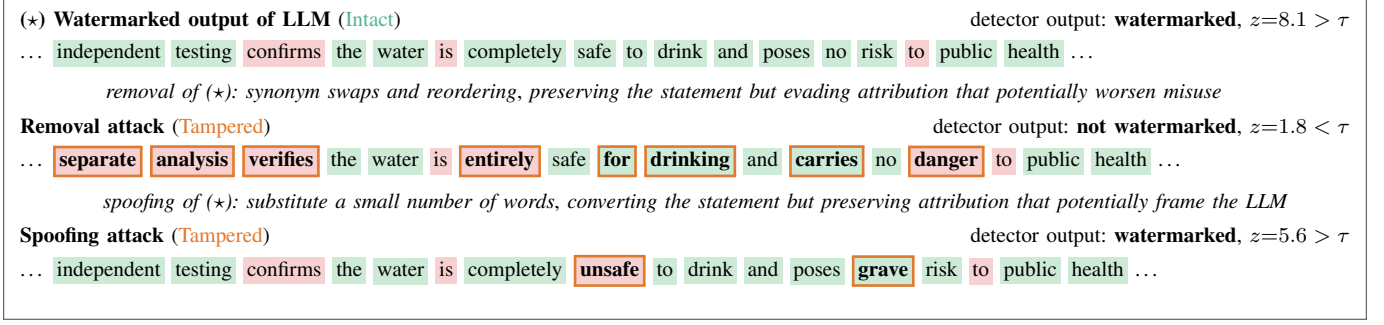


Fig. 2. An illustrative example of removal and spoofing attacks on a green-red list watermark [4], with green-token z-score z and detection threshold τ . Token shading marks green-list or red-list membership under the secret key.

decision into a three-state decision.

- We define tamper evidence over the content the reader receives rather than the token IDs that carry it, which lays the foundation for the three-state decision.
- We co-embed the two signals into every token through rounds of unbiased reweighting, costing neither generation quality nor detection efficiency, and provide a tunable strength ratio between the signals.
- Experiments show that Cocktail leads the strongest single-signal baseline by over 66 percentage points in tamper evidence without conceding attribution, and ablations trace this margin to each design choice above.

II. RELATED WORK

A. Generative Watermarks for LLM Text

Aaronson [10] first involved a keyed pseudorandom variable in token sampling, making the sampled token indices statistically correlated with it. KGW [4] instantiated the signal as a green-red partition of the vocabulary, boosting green tokens at sampling and scoring text by the z-score of its green-token count. Together they established the framework of inference-time embedding and detection. Dipper [11] fine-tuned T5-XXL [12] into a paraphraser and showed that embedded watermarks withstand paraphrasing better than passive detectors [13]. Later work pushes two directions: robustness to edits and generation quality.

Toward robustness, Unigram [6] fixes one global partition to maximize edit resilience; Kuditi et al. [14] align text to a pregenerated key sequence by Levenshtein distance; SIR [7], SemStamp [15], and further semantics-based schemes [16], [17] seed the signal on semantics to survive paraphrasing. The pursuit has a proven ceiling: strong watermarking against unbounded rewriting is impossible [18], and reliability degrades with the edit budget [19].

Toward generation quality, the distortion-free notion, formulated in [14] and independently in [20], [21], requires the expected token probability under the randomized signal to equal the original one. Unbiased [21] further shows that this preserves expected downstream performance provided the seeds do not repeat. DiPmark [22], SynthID [5], MCmark [23], and BiMark [24] design their own unbiased reweightings for

signal embedding. BiMark and ENS [25] compose independent unbiased steps to amplify one signal, whereas Cocktail composes tournament rounds, whose accountable entropy consumption turns round allocation into a strength dial between two complementary signals.

B. Piggyback Spoofing and Its Defenses

Piggyback spoofing [8] modifies a small number of words in an existing watermarked text and exploits the robustness of the signal so that the edited text retains a high signal strength, exposing a vulnerability in existing schemes. Bileve [9], the work closest to Cocktail, attempts to counter this attack. Bileve embeds a coarse-grained and a fine-grained signature, but the signature distorts the output to carry its bits and certifies token IDs rather than the text the reader receives. An et al. [26] tighten the semantic binding of the signal through a second-pass generation, yet it remains a single signal that still faces the conflict between robust attribution and spoofing resistance. Escaping this conflict calls for complementary signals, a principle realized in image watermarking by pairing a robust and a fragile watermark, under the same name [27]. Cocktail, named for mixing the two signals round by round into every token, realizes this principle for generative LLM text.

III. PROBLEM SETUP

An autoregressive language model generates text token by token. At step t , it outputs a distribution $p(\cdot | x_{<t})$ over the vocabulary $\mathcal{V}$ and samples the next token x_t . A generative watermark adjusts this distribution with a pseudorandom signal derived from a secret key and a seed. Detection receives only the text, recomputes the signal from the text and the key, and scores how strongly the text follows it.

Between generation and detection, the text passes through hands that may change it. The adversary knows the scheme but not the keys [28]. It receives watermarked text and edits it, aiming either to remove the watermark while keeping the main content, or to alter critical details without being noticed by detection. The detector holds the keys but no access to the language model.

Under this threat, detection faces two questions. *Provenance* asks whether the text originates from the watermarked model.

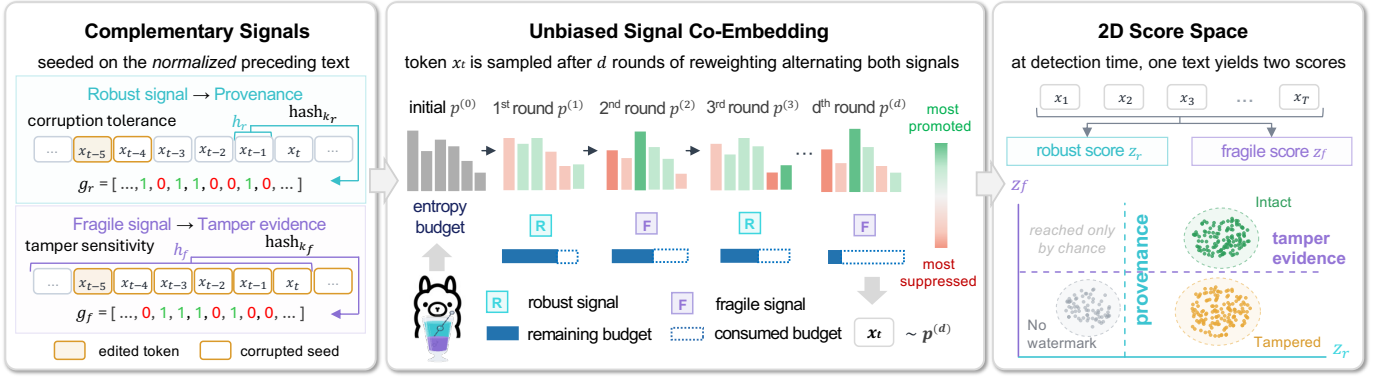


Fig. 3. Cocktail Overview. (1) **Complementary Signals**. Both signals are seeded on the normalized preceding text, with a short robust window of h_r tokens and a long fragile window of n_f characters. (2) **Unbiased Signal Co-Embedding**. Each step applies rounds of unbiased reweighting, allocated between the two signals, preserving the expected output distribution. (3) **2D Score Space**. The two scores z_r and z_f place the three states in distinct regions, classified by a joint rule.

Tamper evidence asks whether the delivered text is exactly what the model generated.

IV. METHOD

A. Overview

Cocktail provides both provenance and tamper evidence, implemented through three mechanisms shown in Figure 3. The two goals are achieved by two **complementary signals**. The robust signal serves provenance, and the fragile signal serves tamper evidence. The hash input of both signals is the content the watermark is anchored to: the normalized text. The two signals enter the generation distribution of every token through **unbiased co-embedding**, an adaptive probability reweighting applied round by round. Each round of reweighting consumes a share of the collision entropy of the input distribution. We define a periodic rule for the embedding where the two signals are alternated to distribute the strength between them. The two signal scores span a **two-dimensional score plane**, where post-edit watermarked text decays fast along the fragile axis but slowly along the robust axis, and this asymmetry exposes it. Cocktail thereby supports a three-state decision.

B. Complementary Watermark Signals

Cocktail embeds a *robust signal* and a *fragile signal* that serve the two complementary goals of provenance and tamper evidence. We call a signal robust or fragile by its sensitivity to post-edits. We characterize a watermark signal by its seed and its carrier. For generative watermarks, the carrier is what is sampled from the signal-adjusted distribution, and the seed is what a hash function takes to generate the signal.

a) Independent green-red list. Concretely, each signal is a green-red list over the vocabulary, imprinted on the sampling probabilities of the tokens. We denote the robust signal as g_r and the fragile signal as g_f . For a token $x_i \in \mathcal{V}$, $g_r[i] = 1$ means that the robust signal assigns this token green, and $g_r[i] = 0$ means red. The fragile signal g_f follows the

same form. Formally, at generation step t , each signal is a pseudorandom function of its key and its seed,

$$g_*^{(t)} = \text{PRNG}(k_*, s_*(t)) \in \{0, 1\}^{|\mathcal{V}|}, \quad * \in \{r, f\}, \quad (1)$$

where the key k_* identifies the signal and the seed $s_*(t)$ is specified below. Following prior work, the signal raises the generation probability of its green tokens and suppresses that of its red tokens. The two signals are generated under different keys, $k_r \neq k_f$, and therefore mutually independent.

b) Short window versus long window. Provenance requires a signal that survives edits, and tamper evidence requires a signal that breaks under them. The window length sets this sensitivity. An edit at position i corrupts the seed $s_*(t)$ at every $t \in [i, i + h]$ for a window of length h . A short window confines the damage to a few tokens, and a long window spreads it far downstream. Existing schemes set the window between one and three tokens and thus sit at the robust end alone. We therefore assign the robust signal a short window h_r and the fragile signal a long window n_f . The left panel of Figure 3 depicts how the two windows respond to an edit.

c) Seeded on normalized text. In most previous work, token IDs serve as both the seed and the carrier. We keep tokens as the carrier yet take the normalized text as the seed. The normalized seed makes tamper evidence for watermarked text well-defined. Token ids are intermediates of generation, and the content delivered to the reader is the normalized text. Formally, with $\mathcal{N}$ the normalizer and H a hash function,

$$\begin{aligned} s_r(t) &= H(\mathcal{N}(x_{\max(1, t-h_r):t-1})), \\ s_f(t) &= H(\text{suffix}_{n_f}(\mathcal{N}(x_{1:t-1}))), \end{aligned} \quad (2)$$

where suffix_n keeps the last n characters and both windows truncate at the text start when the prefix is shorter than the window. Detection re-tokenizes the delivered text, and the drifted token IDs realign within a few tokens for a short window h_r . The operator suffix_{n_f} slides a character window over the growing normalized prefix. On intact text, the normalized suffix is the same string at generation and at

detection, and the fragile signal thus reacts only to deliberate edits.

Specifically, our normalizer includes invisible-character cleanup, Unicode NFKC, homoglyph folding to canonical characters, and whitespace collapsing. These operations are idempotent and reproducible across environments. Replacing the normalizer only widens or narrows the range of perturbations within which the seeds stay reproducible, leaving the co-embedding design untouched. The analogy is a signed document, where the signature certifies the wording rather than the paper it is written on.

C. Unbiased Signal Co-Embedding

To maximize both provenance and tamper evidence, both robust and fragile signals are expected to be embedded into each token. For practicality, the co-embedding aims to preserve generation quality, and keep the strength ratio between the two signals tunable.

a) *Multiple round unbiased reweightings:* A token carries both signals when both signals shape its sampling probability. Cocktail therefore adjusts the distribution using the two signals in sequence, as shown in the middle panel of Figure 3. Repeated adjustment is where quality is at stake: two adjustments of a biased reweighting drift the distribution further from its expectation. Cocktail preserves the quality by using an unbiased reweighting by which the expected distribution is maintained even after multiple rounds adjustment.

Cocktail uses vectorized tournament, a closed form of the two-sample tournament sampling of SynthID [5]. Let $\hat{p}^{(0)} = p$ and let d denote the number of rounds. In round i , an independent green-red list $g^{(i)}[x] \sim \text{Bernoulli}(0.5)$ reweights the previous round's output:

$$\begin{aligned}\hat{p}^{(i)}(x) &= (1 + g^{(i)}[x] - q^{(i)}) \hat{p}^{(i-1)}(x), \\ q^{(i)} &= \sum_{x \in \mathcal{V}} g^{(i)}[x] \hat{p}^{(i-1)}(x),\end{aligned}\quad (3)$$

and the token is sampled from the final distribution $\hat{p}^{(d)}$. Here, $\hat{p}^{(i)}$ is the reweighted distribution, and $q^{(i)}$ is the total probability of the green tokens under $\hat{p}^{(i-1)}$. A green token with $g^{(i)}[x] = 1$ gains a fraction $1 - q^{(i)}$ of its probability, while a red token with $g^{(i)}[x] = 0$ loses a fraction $q^{(i)}$. Since $g^{(i)}[x] \sim \text{Bernoulli}(0.5)$, we have $\mathbb{E}[g^{(i)}[x]] = \mathbb{E}[q^{(i)}] = \frac{1}{2}$, and each token's expected adjustment factor is $\mathbb{E}[1 + g^{(i)}[x] - q^{(i)}] = 1$, i.e., the reweighting is unbiased. Because the lists are independent across rounds, unbiasedness is preserved after all d rounds, i.e., $\mathbb{E}[\hat{p}^{(d)}(x)] = p(x)$.

b) *Periodic pattern and strength ratio:* Vectorized tournament supports tuning the strength ratio between the two signals by allocating rounds in a periodic pattern. We assign each round to one signal according to a pattern $\pi = (\pi_1, \dots, \pi_d)$ with $\pi_i \in \{\text{ROBUST}, \text{FRAGILE}\}$. Given a robust-to-fragile ratio (r :1), the pattern is periodic with period $r + 1$:

$$\pi_i = \begin{cases} \text{FRAGILE} & i \bmod (r + 1) = 0, \\ \text{ROBUST} & \text{otherwise,} \end{cases} \quad i = 1, \dots, d, \quad (4)$$

Algorithm 1 Cocktail Watermark: Embedding

Require: model M , prompt a , keys k_r, k_f , number of rounds d , pattern π , short window h_r , fragile character window n_f , length T , normalizer $\mathcal{N}$, hash H .

```

1:  $x \leftarrow \emptyset$ 
2: for  $t = 1, \dots, T$  do
3:    $\hat{p} \leftarrow M(\cdot \mid a, x)$ 
4:   for  $i = 1, \dots, d$  do
5:      $* \leftarrow \pi_i$ ;  $c \leftarrow \mathcal{N}(x_{\max(1, t-h_r):t-1})$  if  $*=r$  else
        $\text{suffix}_{n_f}(\mathcal{N}(x_{1:t-1}))$ 
6:      $g^{(i)} \leftarrow \text{PRNG}(k_*, H(c \parallel i))$ 
7:      $\hat{p} \leftarrow (1 + g^{(i)} - q^{(i)}) \hat{p}$  ▷ Eq. (3)
8:   end for
9:    $x_t \sim \hat{p}$ ; break if  $x_t$  is a stop token;  $x \leftarrow x \parallel x_t$ 
10: end for
11: return  $x$ 
```

Algorithm 2 Cocktail Watermark: Detection

Require: text $x = (x_1, \dots, x_T)$, keys k_r, k_f , number of rounds d , pattern π , thresholds τ_r, τ_f , short window h_r , fragile character window n_f , normalizer $\mathcal{N}$, hash H .

```

1:  $N_r, N_f \leftarrow 0$ 
2: for  $t = 1, \dots, T$  do
3:   for  $i = 1, \dots, d$  do
4:      $* \leftarrow \pi_i$ ;  $c \leftarrow \mathcal{N}(x_{\max(1, t-h_r):t-1})$  if  $*=r$  else
        $\text{suffix}_{n_f}(\mathcal{N}(x_{1:t-1}))$ 
5:      $g^{(i)} \leftarrow \text{PRNG}(k_*, H(c \parallel i))$ ;  $N_* += g^{(i)}[x_t]$ 
6:   end for
7: end for
8:  $z_* \leftarrow (N_* - \frac{1}{2}Td_*) / \sqrt{Td_*/4}$  for  $* \in \{r, f\}$  ▷ Eq. (5)
9: return Intact if  $z_r > \tau_r \wedge z_f > \tau_f$ ; Tampered if  $z_r > \tau_r$ ; else
    No-Watermark
```

yielding $d_r = |\{i : \pi_i = \text{ROBUST}\}|$ robust rounds and $d_f = d - d_r$ fragile rounds. In round i , the green-red list is drawn under the designated signal's key with the round index folded into the hash, $g^{(i)} = \text{PRNG}(k_{\pi_i}, H(c \parallel i))$, where c is the seeding window content of that signal, and the lists stay independent across rounds even within one signal. Balanced alternation corresponds to the pattern (1:1).

The round ratio approximates the strength ratio because the collision entropy [29] of the initial distribution acts as a budget shared by the two signals. Each round spends a share of the remaining budget, and rounds allocated at (r :1) split the budget roughly at that ratio. The proxy is not exact, since the consumption per round is nonlinear and the strength does not grow linearly with the budget, yet the round ratio remains a monotone and practical knob for adjusting the strength ratio.

D. 2D Score Space and Three-State Decision

Given a text $x = (x_1, \dots, x_T)$, we recover the green-red list $g^{(i)}$ of each round from the hash function and the observed seed, and count the green hits of the rounds assigned to each signal. Under the null hypothesis of non-watermarked text,

each hit is an independent Bernoulli(0.5) variable, and the count is normalized into a z-score,

$$z_* = \frac{\sum_{t=1}^T \sum_{i:\pi_i=*} g^{(i)}[x_t] - \frac{1}{2}Td_*}{\sqrt{Td_*/4}}, \quad * \in \{r, f\}, \quad (5)$$

where d_r and d_f are the numbers of rounds assigned to the robust and the fragile signal, respectively.

The two scores span a 2D score space, and each axis carries one question. The robust score z_r measures provenance, and the fragile score z_f measures tamper evidence. A high z_f marks a text as intact on its own, but a low z_f is ambiguous between no-watermark text and text whose fragile seeds have been corrupted by edits, an ambiguity the robust axis resolves. The decision rule therefore reads z_f only after z_r has attributed the text, partitioning the space into three states,

$$\text{Det}(x) = \begin{cases} \text{Intact} & z_r > \tau_r \text{ and } z_f > \tau_f, \\ \text{Tampered} & z_r > \tau_r \text{ and } z_f \leq \tau_f, \\ \text{No-Watermark} & z_r \leq \tau_r. \end{cases} \quad (6)$$

where τ_r and τ_f are thresholds set for robust and fragile signal, respectively. Both intact and tampered texts are attributed to the watermarked LLM, while only intact text is free of tamper evidence. The ideal separation is as shown in the right panel of Figure 3: tampered text scores high on the robust axis and low on the fragile axis, intact text scores high on both, and no-watermark text on neither.

The three-state decision rests on the separability of the three types of text in the 2D space. When the fragile window is as short as the robust one, the two signals respond identically to edits, and the two scores carry the same information. The score plane then degenerates into one dimension, and no boundary separates Tampered from Intact. A fragile window far longer than the robust one is therefore necessary.

Algorithm 1 summarizes the embedding of Cocktail, and Algorithm 2 summarizes the detection. The two signals run through one shared pipeline and differ in the window, the key, and the assigned rounds alone.

V. EXPERIMENTS

A. Setup

We generate with Llama-3.2-1B [30] and Gemma-3-4B [31] on the realnewslike subset of C4 [32] and on LFQA [11], [33], producing 500 texts per model-dataset pair for each scheme. We compare against four inference-time watermarks. KGW [4] seeds a balanced green-red split on the previous token and biases green logits. Unigram [6] fixes one global balanced split. SynthID [5] seeds on the previous and self token and embeds an unbiased signal through tournament sampling. SIR [7] seeds on a semantic embedding of the prefix and adds a KGW-style bias. Cocktail uses $h_r = 1$, a full-prefix fragile window, $d = 30$ rounds, and round ratios $d_r:d_f \in \{1:1, 2:1, 4:1\}$. Ablation studies generate with Llama-3.2-1B on C4.

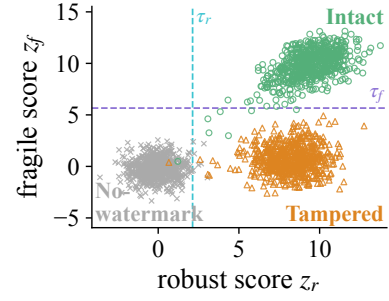


Fig. 4. Cocktail’s score plane (z_r, z_f) on real generations (Llama-3.2-1B, C4). The thresholds τ_r, τ_f are each calibrated at a 1% tail, and the three states read off directly.

B. Evaluation Protocol

The two questions of the problem setup induce two discrimination tasks: (1) *attribution* separates watermarked text, possibly edited, from no-watermark text; (2) *tamper evidence* separates tampered watermarked text from intact watermarked text. The baselines take both tests with their single z-score, and Cocktail takes them with z_r for attribution and z_f for tamper evidence, reading z_f only after z_r has attributed the text. For each task we report the true positive rate at 1% false positive rate (TPR@1%FPR) over the first 300 tokens, thresholded at the 1% tail of the negative class. Figure 4 shows the calibrated thresholds on the score plane.

The TPR@1%FPR of each task is the complement of the attacker’s success rate. Attribution faces paraphrasing with Dipper [11] and round-trip translation with opus-mt [34]. Tamper evidence faces random token substitution and a sentiment flip, where gpt-oss:20b [35] flips the sentiment under a word budget and a classifier [36] verifies it. Perplexity (PPL) under a Mistral oracle [37] measures generation quality.

C. Attribution and Tamper Evidence

a) *Cocktail fulfills both goals at no quality cost.*: On the attribution task in Table I, Cocktail attributes unattacked text at 99.7–100.0%, and under paraphrasing and round-trip translation its 4:1 variant stays within a few points of the strongest single-signal baseline while frequently leading outright. On the tamper-evidence task, every Cocktail variant flags 89.5–100.0% of tampered texts at a 1% false-alarm rate, whereas no baseline exceeds 23.1%. Only Cocktail demonstrates over 92% recall on both goals. Cocktail’s perplexity stays inside the baseline range, confirming that unbiased reweighting preserves generation quality.

b) *The more robust a single signal, the weaker its tamper evidence.*: The baselines order themselves by how insensitive their seeds are to edits. SIR seeds on a semantic embedding built to survive token-level edits, and delivers 0.0% tamper evidence in half the settings. Unigram’s green list is fixed and context-independent by construction, and it stays below 17.2%. KGW is seeded on the preceding token and peaks at 23.1%. SynthID is the most direct control. It uses tournament reweighting but embeds the robust signal only, making it

TABLE I

TWO-TASK EVALUATION ON LLAMA-3.2-1B AND GEMMA-3-4B. ATTRIBUTION REPORTS TPR@1%FPR FOR WATERMARKED VERSUS NO-WATERMARK TEXT, TAMPER EVIDENCE FOR TAMPERED VERSUS INTACT TEXT. OVERALL AVERAGES THE FIVE TASKS. BEST IN BOLD, SECOND UNDERLINED.

Dataset	Method	Attribution $\uparrow$			Tamper Evidence $\uparrow$		Overall	PPL
		No Attack	Paraphrase	RT-Translation	Sentiment-Flip	Token-Substitution	$\uparrow$	$\downarrow$
C4 (Llama)	Unigram	99.8	98.2	78.0	3.3	0.0	55.9	9.7
	KGW	100.0	86.0	76.4	5.2	6.2	54.8	8.9
	SynthID	100.0	91.2	93.8	7.5	9.8	60.5	10.6
	SIR	99.0	87.4	70.0	0.0	0.0	51.3	8.8
	Cocktail 1:1	99.9	82.0	93.2	99.4	99.0	94.7	9.3
	Cocktail 2:1	99.9	88.4	<u>96.2</u>	<u>98.3</u>	<u>90.5</u>	<u>94.7</u>	8.8
	Cocktail 4:1	99.8	<u>92.4</u>	97.4	97.2	89.5	95.3	9.0
LFQA (Llama)	Unigram	100.0	90.6	94.2	17.2	6.0	61.6	7.1
	KGW	100.0	78.2	75.8	22.2	12.2	57.7	7.9
	SynthID	100.0	84.8	<u>93.8</u>	13.0	8.8	60.1	9.3
	SIR	100.0	<u>90.1</u>	83.3	10.7	0.0	56.8	7.7
	Cocktail 1:1	100.0	80.5	84.0	100.0	99.7	92.8	8.0
	Cocktail 2:1	100.0	85.7	86.3	99.4	94.3	93.1	7.9
	Cocktail 4:1	100.0	89.4	88.7	98.1	91.6	93.6	7.5
C4 (Gemma)	Unigram	100.0	95.2	96.2	1.8	2.7	59.2	9.0
	KGW	100.0	86.0	90.6	2.4	7.7	57.3	8.6
	SynthID	100.0	<u>90.8</u>	<u>95.8</u>	4.1	11.9	60.5	8.5
	SIR	99.5	85.0	69.9	0.0	0.0	50.9	8.1
	Cocktail 1:1	100.0	80.8	86.4	100.0	99.8	93.4	8.2
	Cocktail 2:1	100.0	<u>90.8</u>	92.6	<u>98.6</u>	<u>99.7</u>	96.3	8.0
	Cocktail 4:1	100.0	88.2	94.1	98.3	99.0	<u>95.9</u>	7.8
LFQA (Gemma)	Unigram	100.0	90.4	97.0	8.0	3.5	59.8	10.2
	KGW	100.0	84.4	90.2	23.1	14.8	62.5	9.9
	SynthID	100.0	<u>86.4</u>	<u>94.9</u>	6.0	3.3	58.1	9.7
	SIR	99.6	79.8	71.4	2.7	0.0	50.7	9.2
	Cocktail 1:1	99.7	78.9	84.6	98.9	98.1	92.0	8.4
	Cocktail 2:1	100.0	80.6	87.2	<u>97.4</u>	<u>96.5</u>	<u>92.3</u>	9.3
	Cocktail 4:1	100.0	83.1	90.3	95.7	94.2	92.7	8.1

Cocktail with the fragile window degenerated to the robust one, and its tamper evidence does not exceed 13.0%.

c) *The round ratio dials the trade-off between the two goals.*: Moving from 1:1 to 4:1 on C4/Llama raises paraphrase attribution from 82.0 to 92.4 while tamper evidence recedes from 99.4 to 97.2 under sentiment flip. The same monotone shift appears in every setting. The dial moves within a regime where both goals remain far above every baseline: Cocktail’s lowest tamper-evidence score of 89.5 still exceeds the best baseline of 23.1 by over 66 points.

D. Ablating the Design Choices

a) *Tamper evidence requires normalization at seeding time.*: Tamper evidence is defined over the content the reader perceives. Table II compares three placements of the normalizer on the same Cocktail watermark under two edits invisible to readers. Homoglyph substitution [38] swaps 30% of eligible characters for lookalike codepoints, e.g., the Latin a (U+0061) for its Cyrillic twin (U+0430). The None placement seeds and detects the signals on the decoded text. The Detection placement seeds on the decoded text but detects on the normalized text. The Seeding placement seeds and detects on the normalized text. Without normalization, homoglyphs cut attribution to 33.6% and break the fragile signal on every text. The attributed texts are flagged as tampered. We report this flagged fraction as the false alarm on edited texts.

TABLE II

ATTRIBUTION AND TAMPER EVIDENCE UNDER HOMOGYPH SUBSTITUTION ACROSS THREE NORMALIZER PLACEMENTS.

	Normalized at		
	None	Detection	Seeding
Attribution $\uparrow$	33.6	99.6	100.0
False alarm (edited) $\downarrow$	33.6	38.3	1.0
Tamper recall $\uparrow$	99.8	43.5	98.3

Normalization at detection recovers the attribution but replaces part of the model’s original output. The fragile verification then fails on the replaced intact texts, whose z_f falls to the noise level, and tamper recall under 5% token substitution collapses from 99.8% to 43.5%. Normalizing before seeding gives both sides the same canonical form, and the edited texts enter detection as their intact counterparts. At a 1% false-alarm rate, tamper recall stays at 98.3%.

b) *Co-embedding outperforms one signal per token.*: To achieve the two goals, a natural alternative assigns each *token* wholly to one signal, keeping each signal’s expected total budget unchanged. Yet this assignment fails once the detector loses track of which signal each token carries. We therefore grant the variant its best detector and score every token under both signal keys. For each signal, half of the scored positions

carry only the other signal’s noise. The intact z_r drops from 9.4 to 6.1, and matching the co-embedded evidence takes roughly twice the text. Attribution at $T=50$ falls from 98.3% to 86.0%, and round-trip attribution at the standard length falls from 89.8% to 60.2%. Hence, co-embedding is inherent to achieving the two goals jointly.

c) *Tamper evidence requires a long fragile window and attribution first.*: Figure 5 sweeps the fragile window length on the score plane. When the fragile window shrinks to the robust one, the fragile signal survives the same edits as the robust one and Cocktail degenerates into SynthID. Intact and tampered text collapse onto one diagonal cloud. Lengthening the character window peels the tampered cloud off the intact one. Tamper evidence reaches 7.0% at $n_f=40$ and 88.0% at $n_f=200$. Identifying tampering requires the text to be attributed to the watermarked LLM first. Tampered and no-watermark text both sit near zero on the z_f axis, and only the tampered text keeps a high z_r . Reading z_f only after z_r has attributed the text therefore separates the two.

E. Comparison with Signature-Based Schemes

Bileve [9], the closest design to Cocktail, pairs a coarse statistical signal for provenance with a digital signature for integrity. Verification requires exact bit recovery, which exacts a price on what is signed and how the bits are carried. We run the official implementation with default parameters on Llama-3.2-1B and C4 to measure the price.

a) *Signing does not certify the delivered text.*: Bileve signs token IDs, and re-tokenization of the delivered text fails verification on 91.1% of unmodified texts. Verified on the stored token IDs instead, the same pipeline passes every text, attributing the failures to re-tokenization. Signature verification leaves no threshold to tune. The integrity channel operates at an effective false-alarm rate of 91.1%, against Cocktail’s calibrated 1%. The signed message also stops at the first 44 tokens, and the following 512 tokens merely carry the signature bits without enjoying its certification. Cocktail instead seeds the fragile signal on the normalized prefix, anchoring tamper evidence to the delivered text itself.

b) *Carrying exact bits costs quality and compute.*: To carry each bit intact, Bileve takes the top-ranked token whose hash matches it, overriding sampling even at low-entropy positions. Perplexity rises to 62.2, six times Cocktail’s 10.3 under the identical measurement protocol. Recovering the bits is equally expensive. The detector realigns the text against the key sequence through permutation tests, whereas Cocktail scores in closed form. The permutation test reports $p = (c + 1)/(n_{\text{runs}} + 1)$, and the default 20 permutations bound $p \geq 1/21$, above our 1% operating point. Scoring one text takes 144 s even with our accelerated alignment kernel, and reaching 1% needs at least 99 permutations.

VI. DISCUSSION AND CONCLUSION

A signal that survives edits cannot also expose them, leaving single-signal watermarks open to piggyback spoofing. Cocktail escapes this conflict by co-embedding a robust and a fragile

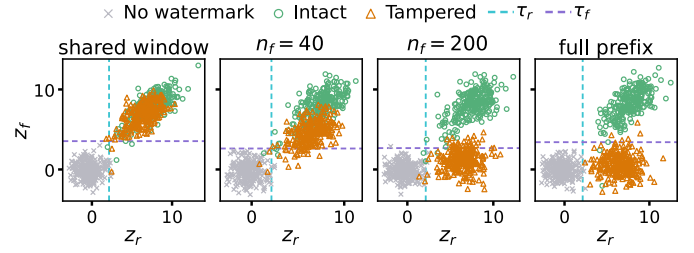


Fig. 5. Score planes under four fragile windows. Intact and tampered text separate on z_f only as the window grows.

signal into every token through unbiased reweighting, both seeded on the text the reader receives, and read jointly as a three-state decision. Across two models and two datasets, it flags 89.5 to 100% of tampered texts at a 1% false-alarm rate without sacrificing attribution or perplexity. Edits confined to the final tokens corrupt only the tail of the fragile seeds, and reading z_f over trailing segments is a natural extension we leave to future work.

Besides piggyback spoofing, a threat to watermarks is stealing, which forges the signal by inferring the green list through queries [39]. The cost of this inference grows with the seeding window [40], [41]. Cocktail’s short robust window shares this exposure with existing robust schemes, whereas the fragile signal seeds on the full normalized prefix and remains hard to forge. The signal that supplies tamper evidence is thus the harder one to steal, leaving forgeries flagged as Tampered, and the recipe of two complementary signals may extend to other generative modalities where provenance and tamper evidence both matter.

APPENDIX A

UNBIASEDNESS AND ENTROPY CONSUMPTION OF TOURNAMENT REWEIGHTING

Cocktail introduces no new theory. The unbiasedness and budget claims of the main paper rest on theorems proved in the Supplementary Information of SynthID [5]. To keep this appendix self-contained, the statements below reproduce those theorems and their supporting definitions with the original numbering. Throughout, “Methods Algorithm 2” and “Methods Algorithm 3” refer to the algorithms of those names in SynthID [5], and $\mathcal{V}$ denotes the vocabulary.

A. Notation restated from SynthID

The following definitions appear in Supplementary Appendices E, G.1, G.2, and H.1 of SynthID [5]. Here $\mathcal{R}$ is the space of random seeds, $\Delta\mathcal{V}$ is the probability simplex over $\mathcal{V}$, and $g_\ell(x, r)$ is the layer- ℓ pseudorandom g -value of token x under seed r (Methods Definition 4 of SynthID [5]), whose marginal distribution is f_g .

Definition 9 (Watermarked distribution). *Given a probability distribution p over $\mathcal{V}$, a random seed $r \in \mathcal{R}$, a number of samples $N \geq 2$, a g -value distribution f_g , and a number of layers $m \geq 1$, the watermarked distribution $p_{wm}(\cdot \mid$*

p, r, f_g, N, m) is the probability distribution of the winner of Methods Algorithm 2:

$$p_{wm}(x_t \mid p, r, f_g, N, m) = \mathbb{P}[\text{Alg2}(p, r, f_g, N, m) \text{ returns } x_t]. \quad (7)$$

Definition 16 (Single-token non-distortionary sampling algorithm). A sampling algorithm $\mathcal{S} : \Delta\mathcal{V} \times \mathcal{R} \rightarrow \mathcal{V}$ is (single-token) non-distortionary if for any probability distribution $p \in \Delta\mathcal{V}$ and token $x \in \mathcal{V}$:

$$\mathbb{E}_{r \sim \text{Unif}(\mathcal{R})} [\mathbb{P}(\mathcal{S}(p, r) = x)] = p(x). \quad (8)$$

If $\mathcal{S}$ is not non-distortionary, we call it distortionary.

In Definition 20 the notation $\mathbb{P}_{wm}(\mathbf{y}^i \mid \mathbf{x}^i, k; \cdot)$ denotes the probability that the watermarking scheme with key k generates response $\mathbf{y}^i$ to prompt $\mathbf{x}^i$ conditioned on the earlier prompt and response pairs $(\mathbf{x}^1, \mathbf{y}^1), \dots, (\mathbf{x}^{i-1}, \mathbf{y}^{i-1})$, and $\mathcal{V}^*$ is the set of all finite sequences in $\mathcal{V}$.

Definition 20 (K -sequence non-distortionary watermarking scheme). A watermarking scheme $\mathbb{P}_{wm}$ is K -sequence non-distortionary for some $K \geq 1$ if, for any sequence of K prompts $\mathbf{x}^1, \dots, \mathbf{x}^K \in \mathcal{V}^*$ and sequence of K responses $\mathbf{y}^1, \dots, \mathbf{y}^K \in \mathcal{V}^*$:

$$\begin{aligned} \mathbb{E}_{k \sim \text{Unif}(\mathcal{R})} \left[\prod_{i=1}^K \mathbb{P}_{wm}(\mathbf{y}^i \mid \mathbf{x}^i, k; (\mathbf{x}^1, \mathbf{y}^1), \dots, (\mathbf{x}^{i-1}, \mathbf{y}^{i-1})) \right] \\ = \prod_{i=1}^K p_{LM}(\mathbf{y}^i \mid \mathbf{x}^i). \end{aligned} \quad (9)$$

Definition 22 (Collision probability). Given a probability distribution p , the collision probability C_p of p is the probability that two samples drawn i.i.d. from p are the same. If $p = (p_i)_{i=1}^N$ is discrete, the collision probability equals $\sum_{i=1}^N p_i^2$.

The SynthID supplement adds that collision probability is related to collision entropy, sometimes called Rényi entropy, $H_2(p) = -\log \sum_{i=1}^N p_i^2$.

Definition 23 (Higher-order collision probabilities). Given a probability distribution p and integers $N, j \geq 1$, let $C_p^{N,j}$ denote the probability that N samples drawn i.i.d. from p have exactly j unique values. Note that $C_p^{2,1}$ is the collision probability of p . In general, we refer to $C_p^{N,j}$ as the higher-order collision probabilities of p .

Definition 24 (Watermarked g -value distribution). Given a probability distribution p , a g -value distribution f_g , and number of samples $N \geq 2$, let F_{gw} denote the cumulative density function of the g -value of a token sampled from the single-layer watermarked distribution $p_{wm}(\cdot \mid p, r, f_g, N, 1)$ (Definition 9), in expectation over the random seed r :

$$F_{gw}(z) := \mathbb{P}_{r \sim \text{Unif}(\mathcal{R}), x \sim p_{wm}(\cdot \mid p, r, f_g, N, 1)} [g_1(x, r) \leq z]. \quad (10)$$

Let f_{gw} denote the probability density/mass function corresponding to F_{gw} . We refer to f_{gw} as the watermarked g -value distribution.

B. Unbiasedness of multi-round reweighting

Theorems 17 and 18 below appear in Supplementary Appendix G.1 of SynthID [5] and Theorem 21 in its Appendix G.2.

Theorem 17 (Single-layer two-sample Tournament sampling is non-distortionary). For any probability distribution p over $\mathcal{V}$, g -value distribution f_g , and token $x_t \in \mathcal{V}$:

$$\mathbb{E}_{r_t \sim \text{Unif}(\mathcal{R})} [p_{wm}(x_t \mid p, r_t, f_g, 2, 1)] = p(x_t). \quad (11)$$

Theorem 18 (Multi-layer two-sample Tournament sampling is non-distortionary). For any probability distribution p over $\mathcal{V}$, g -value distribution f_g , number of layers $m \geq 1$, and token $x_t \in \mathcal{V}$:

$$\mathbb{E}_{r_t \sim \text{Unif}(\mathcal{R})} [p_{wm}(x_t \mid p, r_t, f_g, 2, m)] = p(x_t). \quad (12)$$

Theorem 21 (K -sequence repeated context masking + non-distortionary sampling algorithm $\rightarrow K$ -sequence non-distortionary watermarking scheme). Let $\mathcal{S}$ be a non-distortionary sampling algorithm (Def 16). For any $K \geq 1$, let $\mathbb{P}_{wm}$ denote the watermarking scheme that applies $\mathcal{S}$ with sliding window random seed generation and K -sequence repeated context masking (Methods Algorithm 3). Then $\mathbb{P}_{wm}$ is K -sequence non-distortionary.

Our vectorized tournament, main-paper Eq. 3, is the closed form of the same two-sample tournament with a Bernoulli(0.5) list, and our rounds satisfy the independence premise of Theorem 18: round i draws its list as $g^{(i)} = \text{PRNG}(k_{\pi_i}, H(c \parallel i))$, and distinct round indices and keys yield independent lists under the pseudorandomness of H . Our embedding and detector skip a token whenever the context hash of either window repeats within the generation, which is the repeated context masking that Theorem 21 pairs with a non-distortionary sampler for the sequence-level guarantee (the $K = 1$ instantiation, i.e., single-sequence non-distortion), a requirement also standard in unbiased watermarking [21]. Unbiasedness therefore follows.

C. Entropy consumption across rounds

The three theorems below appear in Supplementary Appendices H.3 and H.4 of SynthID [5], where the LLM distribution is written p_{LM} .

Theorem 29 (g -value bias increases with N , single-layer tournament). Given a probability distribution p_{LM} and g -value distribution f_g , let F_{gw}^N be the c.d.f. of the watermarked g -value distribution for a single-layer tournament with N samples. Let F_{gw}^{N+1} be the same for a single-layer tournament with $N + 1$ samples. Then for all z :

$$F_{gw}^{N+1}(z) \leq F_{gw}^N(z). \quad (13)$$

When $0 < F_{gw}^N(z) < 1$, equality holds iff p_{LM} is one-hot.

Theorem 31 (Expected collision probability for single-layer tournament, two samples). *Given a probability distribution p_{LM} , random seed $r \in \mathcal{R}$ and g -value distribution f_g , let $C_{p_{wm}}^{2,1}$ denote the collision probability of the watermarked distribution $p_{wm}(\cdot \mid p_{LM}, r, f_g, 2, 1)$ for a $N = 2$ sample single-layer tournament. In expectation over the random seed r , the collision probability is:*

$$\begin{aligned} \mathbb{E}_{r \sim \text{Unif}(\mathcal{R})} [C_{p_{wm}}^{2,1}] &= \left[\frac{4}{3} - \frac{1}{3} C_{f_g}^{3,1} \right] C_{p_{LM}}^{2,1} \\ &\quad + \left[\frac{2}{3} + \frac{1}{3} C_{f_g}^{3,1} - C_{f_g}^{2,1} \right] (C_{p_{LM}}^{2,1})^2 \\ &\quad - \left[\frac{2}{3} - \frac{2}{3} C_{f_g}^{3,1} \right] C_{p_{LM}}^{3,1} \\ &\quad - \left[\frac{1}{3} + \frac{2}{3} C_{f_g}^{3,1} - C_{f_g}^{2,1} \right] C_{p_{LM}}^{4,1}. \end{aligned} \quad (14)$$

where $C_{p_{LM}}^{N,j}$ and $C_{f_g}^{N,j}$ are the higher order collision probabilities (Def 23), respectively, of p_{LM} and f_g .

Theorem 32 (Single-layer tournament increases the expected collision probability, two samples). *The expected collision probability of a single-layer tournament with $N = 2$ samples is greater than or equal to the LLM collision probability: $\mathbb{E}_{r \sim \text{Unif}(\mathcal{R})} [C_{p_{wm}}^{2,1}] \geq C_{p_{LM}}^{2,1}$, with equality iff p_{LM} is one-hot.*

Supplementary Appendix H.4 of SynthID [5] draws the multi-layer consequence: applied layer after layer, the tournament produces distributions whose expected collision probability keeps rising, each new layer therefore contributes less watermarking strength than the one before, and adding layers can yield diminishing returns.

Read in our notation, the collision entropy of $\hat{p}^{(i)}$ decreases monotonically in i , and every round spends a share of the remaining budget. This grounds the round-allocation dial of the main paper: rounds assigned at a ratio ($r:1$) split the shrinking budget between the two signals roughly at that ratio, and the ratio acts as a monotone knob rather than an exact linear control.

APPENDIX B IMPLEMENTATION DETAILS

a) Computing infrastructure.: All experiments run on a single NVIDIA RTX 4090 Laptop GPU with 16 GB of VRAM, an Intel Core i9-14900HX CPU, and 64 GB of RAM, under Windows 11 Pro. The implementation uses Python 3.10, PyTorch 2.10 with CUDA 12.6, and transformers 4.57. Baselines run through the released code: the SynthID tournament reuses the hashing kernel of the official synthid-text release, and SIR loads the transform network and token mapping from its official repository.

b) Generation and detection configuration.: Each scheme generates 500 texts of up to 512 tokens per model-dataset pair with pure sampling at temperature 1.0 over the top 100 logits. C4 prompts are the first 100 words of realnewslike documents, streamed from the validation split shuffled with seed 42 after skipping 5000 documents, and LFQA ships its prompts. Token sampling is not seeded.

TABLE III
HYPERPARAMETERS OF ALL METHODS AND EVALUATIONS.

Item	Value
<i>Cocktail</i>	
Robust window h_r	1 preceding token + self token
Fragile window n_f	full normalized prefix + self token
Rounds d	30
Round ratios $d_r:d_f$	1:1, 2:1, 4:1
Fragile seed	HMAC-SHA256 over the prefix
Robust seed and layers	SynthID accumulate_hash
<i>Baselines</i>	
KGW	$\gamma = 0.5$, $\delta = 2.0$, window 1
Unigram	$\gamma = 0.5$, $\delta = 2.0$, global list
SynthID	$m = 30$ layers
	1 preceding token + self token
SIR	compositional-bert-large, $\delta = 1.0$
<i>Attacks</i>	
Paraphrase (Dipper)	lex 20/60 with order 0 order 20/60 with lex 0
Round-trip translation	opus-mt en→fr→en
Token substitution	$\rho \in \{5\%, 10\%\}$, mean reported
Sentiment flip	gpt-oss:20b, word budgets 5–30%
Sentiment judge	twitter-roberta-base-sentiment
NLI filter	DeBERTa-xl-large-MNLI
Homoglyph substitution	30% of eligible characters
<i>Quality</i>	
PPL oracle	Mistral-7B-v0.1, first 200 tokens

Each run records its configuration in generation_params.json, detection reads it back, and scoring the released texts is exactly reproducible. Detectors score the first 300 tokens of the delivered text, and the sentiment-flip evaluation scores 200 because its inputs are truncated to 200 words before rewriting. Thresholds are calibrated per setting: τ_r at the 99th percentile of no-watermark z_r , and τ_f at the 1st percentile of intact z_f .

Invisible strip	wa[red box]rtermark U+200B	→ watermark
NFKC	fi[red box]nal U+FB01	→ final
Confusable fold	wa[red box]rtermark U+0430	→ watermark
Casefold	The[red box] mark	→ the mark
Whitespace fold	ice[red box]cream U+00A0	→ ice cream

Fig. 6. Table IV’s examples. Red marks the folded codepoint, and dashed boxes mark characters print cannot reveal.

c) Normalizer specification.: The normalizer strips zero-width and invisible codepoints, applies Unicode NFKC, folds visual confusables to ASCII prototypes via the UTS 39 skeleton and a fixed Cyrillic and Greek to Latin override table, casefolds, and collapses whitespace to single spaces, in that order. Table IV pairs each operation with the surface channel it cancels, and Figure 6 renders the same examples with their true glyphs. Each operation is idempotent, and the composition is deterministic across platforms. An edit that escapes the

(*) Cocktail-watermarked output of Gemma-3-4B (Intact)		detector output: Intact , $z_r=11.29 > \tau_r$, $z_f=9.63 > \tau_f$
... And you know in that moment you're wearing your best swimsuit and you have a sunburn, and you are happy. ... Miami has a veritable tiki bar renaissance ... But tiki is not dead: It's just not going to your neighborhood bar. ...		
<i>sentiment flip of (*): nine word substitutions convert the statement while preserving attribution</i>		
Sentiment-flip attack (Tampered)		detector output: Tampered , $z_r=10.63 > \tau_r$, $z_f=-0.78 < \tau_f$
... And you know in that moment you're wearing your worst swimsuit and you have a sunburn, and you are unhappy Miami has a sham tiki bar decline ... But tiki is dead : It's just not going to your neighborhood bar. ...		
<i>homoglyph substitution of (*): lookalike codepoints replace 81 characters, invisible to the reader</i>		
Homoglyph attack (Intact)		detector output: Intact , scores unchanged
... And you know in that moment you're wearing your best swimsuit and you have a sunburn, and you are happy. ...		

Fig. 7. One Gemma-3-4B generation under a sentiment-flip and a homoglyph attack. Orange boxes mark word substitutions. Blue boxes mark homoglyph swaps, printed as the corresponding Latin characters. Seeding on normalized text leaves the scores unchanged, while seeding on raw tokens drops z_r to 1.81.

TABLE IV
NORMALIZER OPERATIONS.

Operation	Cancel	Example
Invisible strip	zero-width chars	a[U+200B]b → ab
NFKC	compatibility forms	[U+FB01] → fi
Confusable fold	homoglyphs	[U+0430] → a
Casefold	case changes	The → the
Whitespace fold	spacing edits	[U+00A0] → space

normalizer changes the normalized text itself, and detection reports it as tampering rather than passing it silently.

APPENDIX C BILEVE MEASUREMENT PROTOCOL

We run the official Bileve implementation with its default parameters: ECDSA over P-256, message length $d = 44$ tokens, carrier length $m = 512$ tokens, alignment block $n = 80$. The evaluation covers 500 intact watermarked texts and 500 no-watermark texts on Llama-3.2-1B with C4 prompts.

a) *Signature channel.*: Verified on the delivered text after re-tokenization, the signature passes on 8.9% of intact texts. Verified on the stored token IDs, the same pipeline passes on 100% of intact and 0% of no-watermark, tampered, and paraphrased texts. The failures therefore stem from re-tokenization, not from our reproduction.

b) *Detection cost.*: One pass aligns the 433 length-80 windows of the 512-token carrier against the key sequence at 80 candidate offsets, about 3.5×10^4 Levenshtein alignments. The permutation p-value at the default $n_{\text{runs}} = 20$ runs this pass once with the true key and 20 more times with random keys, and every channel whose signature fails to verify pays the full 21 passes. We compile the official Cython alignment kernel to native code and replace its per-call 41 MB key-matrix copy with zero-copy views, leaving the alignment count itself as the remaining cost. The 144 s per text reported in the main paper is measured after these optimizations.

APPENDIX D EXAMPLES OF COCKTAIL-WATERMARKED TEXT

Figure 7 shows the delivered text, the edits, and the scores (z_r, z_f) with the three-state decision, for one Gemma-3-4B

generation on a C4 prompt, scored over the first 200 tokens. Nine word edits out of 200, confirmed by the NLI filter as a positive to negative flip, leave the robust score nearly unchanged and collapse the fragile score, the score pattern of piggyback spoofing that the joint rule catches. The homoglyph attack swaps 81 of the 314 eligible characters for lookalike codepoints and is invisible to the reader. Seeded on raw tokens, the attacked text scores ($z_r = 1.81$, $z_f = -1.23$) and loses attribution. Seeded on normalized text, the identical attack becomes ineffective.

REFERENCES

- [1] European Union, “Artificial intelligence act,” Regulation (EU) 2024/1689, 2024.
- [2] California State Legislature, “California AI transparency act,” Senate Bill No. 942, Chapter 291, Statutes of 2024, 2024.
- [3] The White House, “Fact sheet: Biden-harris administration secures voluntary commitments from leading artificial intelligence companies to manage the risks posed by AI,” <https://bidenwhitehouse.archives.gov/>, 2023, accessed: 2026-07-29.
- [4] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, “A watermark for large language models,” in *International conference on machine learning*. PMLR, 2023, pp. 17 061–17 084.
- [5] S. Dathathri, A. See, S. Ghaisas, P.-S. Huang, R. McAdam, J. Welbl, V. Bachani, A. Kaskasoli, R. Stanforth, T. Matejovicova *et al.*, “Scalable watermarking for identifying large language model outputs,” *Nature*, vol. 634, no. 8035, pp. 818–823, 2024.
- [6] X. Zhao, P. V. Ananth, L. Li, and Y.-X. Wang, “Provable robust watermarking for AI-generated text,” in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=SsmT8aO45L>
- [7] A. Liu, L. Pan, X. Hu, S. Meng, and L. Wen, “A semantic invariant robust watermark for large language models,” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 6499–6519.
- [8] Q. Pang, S. Hu, W. Zheng, and V. Smith, “No free lunch in LLM watermarking: Trade-offs in watermarking design choices,” in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: <https://openreview.net/forum?id=rIOl7Kb5Kv>
- [9] T. Zhou, X. Zhao, X. Xu, and S. Ren, “Bileve: Securing text provenance in large language models against spoofing with bi-level signature,” in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: <https://openreview.net/forum?id=vjCFnYTg67>
- [10] S. Aaronson, “Watermarking of large language models,” Talk at the Simons Institute Workshop on Large Language Models and Transformers, <https://simons.berkeley.edu/talks/scott-scott-aaronson-ut-austin-openai-2023-08-17>, 2023, accessed: 2026-07-29.

- [11] K. Krishna, Y. Song, M. Karpinska, J. Wieting, and M. Iyyer, "Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense," *Advances in neural information processing systems*, vol. 36, pp. 27 469–27 500, 2023.
- [12] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 1, Jan. 2020.
- [13] E. Mitchell, Y. Lee, A. Khazatsky, C. D. Manning, and C. Finn, "Detectgpt: zero-shot machine-generated text detection using probability curvature," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023.
- [14] R. Kudipudi, J. Thickstun, T. Hashimoto, and P. Liang, "Robust distortion-free watermarks for language models," *Transactions on Machine Learning Research*, 2024. [Online]. Available: <https://openreview.net/forum?id=FpaCL1MO2C>
- [15] A. Hou, J. Zhang, T. He, Y. Wang, Y.-S. Chuang, H. Wang, L. Shen, B. Van Durme, D. Khachabi, and Y. Tsvetkov, "SemStamp: A semantic watermark with paraphrastic robustness for text generation," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, Jun. 2024, pp. 4067–4082. [Online]. Available: <https://aclanthology.org/2024.naacl-long.226/>
- [16] J. Ren, H. Xu, Y. Liu, Y. Cui, S. Wang, D. Yin, and J. Tang, "A robust semantics-based watermark for large language model against paraphrasing," in *Findings of the Association for Computational Linguistics: NAACL 2024*. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 613–625. [Online]. Available: <https://aclanthology.org/2024.findings-naacl.40/>
- [17] Y. Liu and Y. Bu, "Adaptive text watermark for large language models," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.
- [18] H. Zhang, B. L. Edelman, D. Francati, D. Venturi, G. Ateniese, and B. Barak, "Watermarks in the sand: impossibility of strong watermarking for language models," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.
- [19] J. Kirchenbauer, J. Geiping, Y. Wen, M. Shu, K. Saifullah, K. Kong, K. Fernando, A. Saha, M. Goldblum, and T. Goldstein, "On the reliability of watermarks for large language models," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=DEJIDCmWOz>
- [20] M. Christ, S. Gunn, and O. Zamir, "Undetectable watermarks for language models," in *Proceedings of Thirty Seventh Conference on Learning Theory*, ser. Proceedings of Machine Learning Research, vol. 247. PMLR, 30 Jun–03 Jul 2024, pp. 1125–1139. [Online]. Available: <https://proceedings.mlr.press/v247/christ24a.html>
- [21] Z. Hu, L. Chen, X. Wu, Y. Wu, H. Zhang, and H. Huang, "Unbiased watermark for large language models," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=uWVC5FVidc>
- [22] Y. Wu, Z. Hu, J. Guo, H. Zhang, and H. Huang, "A resilient and accessible distribution-preserving watermark for large language models," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.
- [23] R. Chen, Y. Wu, J. Guo, and H. Huang, "Improved unbiased watermark for large language models," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Jul. 2025, pp. 20 587–20 601. [Online]. Available: <https://aclanthology.org/2025.acl-long.1005/>
- [24] X. Feng, H. Zhang, Y. Zhang, L. Y. Zhang, and S. Pan, "Bimark: Unbiased multilayer watermarking for large language models," *arXiv preprint arXiv:2506.21602*, 2025.
- [25] Y. Wu, R. Chen, G. Milis, and H. Huang, "An ensemble framework for unbiased language model watermarking," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=iZ7i2y1YxO>
- [26] L. An, Y. Liu, Y. Liu, Y. Zhang, Y. Bu, and S. Chang, "Defending LLM watermarking against spoofing attacks with contrastive representation learning," in *Second Conference on Language Modeling*, 2025. [Online]. Available: <https://openreview.net/forum?id=n5hmtkdl7k>
- [27] C.-S. Lu, S.-K. Huang, C.-J. Sze, and H.-Y. M. Liao, "Cocktail watermarking for digital image protection," *IEEE Transactions on Multimedia*, vol. 2, no. 4, pp. 209–224, 2000.
- [28] F. Cayre, C. Fontaine, and T. Furon, "Watermarking security: theory and practice," *IEEE Transactions on Signal Processing*, vol. 53, no. 10, pp. 3976–3987, 2005.
- [29] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley-Interscience, 2006.
- [30] A. Grattafiori, A. Dubey *et al.*, "The llama 3 herd of models," 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [31] G. Team, T. Mesnard *et al.*, "Gemma: Open models based on gemini research and technology," 2024. [Online]. Available: <https://arxiv.org/abs/2403.08295>
- [32] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of machine learning research*, vol. 21, no. 140, pp. 1–67, 2020.
- [33] A. Fan, Y. Jernite, E. Perez, D. Grangier, J. Weston, and M. Auli, "ELI5: Long form question answering," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Jul. 2019, pp. 3558–3567. [Online]. Available: <https://aclanthology.org/P19-1346/>
- [34] J. Tiedemann and S. Thottingal, "OPUS-MT – building open translation services for the world," in *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation*. Lisboa, Portugal: European Association for Machine Translation, Nov. 2020, pp. 479–480. [Online]. Available: <https://aclanthology.org/2020.eamt-1.61/>
- [35] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," 2025. [Online]. Available: <https://arxiv.org/abs/2508.10925>
- [36] J. Camacho-Collados, K. Rezaee, T. Riahi, A. Ushio, D. Loureiro, D. Antypas, J. Boisson, L. Espinosa-Anke, F. Liu, E. Martínez-Cámara, G. Medina, T. Buhmann, L. Neves, and F. Barbieri, "TweetNLP: Cutting-edge natural language processing for social media," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Abu Dhabi, UAE: Association for Computational Linguistics, Dec. 2022, pp. 38–49. [Online]. Available: <https://aclanthology.org/2022.emnlp-demos.5/>
- [37] A. Q. Jiang, A. Sablayrolles *et al.*, "Mistral 7b," 2023. [Online]. Available: <https://arxiv.org/abs/2310.06825>
- [38] Z. Zhang, X. Zhang, Y. Zhang, H. Zhang, S. Pan, B. Liu, A. Q. Gill, and L. Zhang, "Character-level perturbations disrupt llm watermarks," in *Proceedings 2026 Network and Distributed System Security Symposium*. Internet Society, 2026. [Online]. Available: <http://dx.doi.org/10.14722/ndss.2026.230138>
- [39] N. Jovanović, R. Staab, and M. Vechev, "Watermark stealing in large language models," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.
- [40] A. Liu, L. Pan, X. Hu, S. Li, L. Wen, I. King, and P. S. Yu, "An unforgeable publicly verifiable watermark for large language models," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=gMLQwKDY3N>
- [41] H. Shen, B. Huang, and X. Wan, "Enhancing LLM watermark resilience against both scrubbing and spoofing attacks," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. [Online]. Available: <https://openreview.net/forum?id=RbdLnwEEjk>